



CENTRO UNIVERSITÁRIO DE BRASÍLIA
FACULDADE DE TECNOLOGIA E CIÊNCIAS SOCIAIS APLICADAS
CURSO DE CIÊNCIA DA COMPUTAÇÃO
PROJETO INTEGRADOR I — TURMA C

Professora Orientadora: Kadidja Valéria Reginaldo de Oliveira

TUTORIAL DE INSTALAÇÃO E EXECUÇÃO DO PROTÓTIPO MAPPA

Guia simples para baixar o projeto no GitHub, executar o ETL, iniciar o dashboard em Streamlit e entender o funcionamento geral do sistema.

GRUPO:
VICTOR FERNANDES
JOÃO VITOR VELOZO
LUCAS MONTEIRO
IGOR CAVALCANTE
DANILO LISBOA

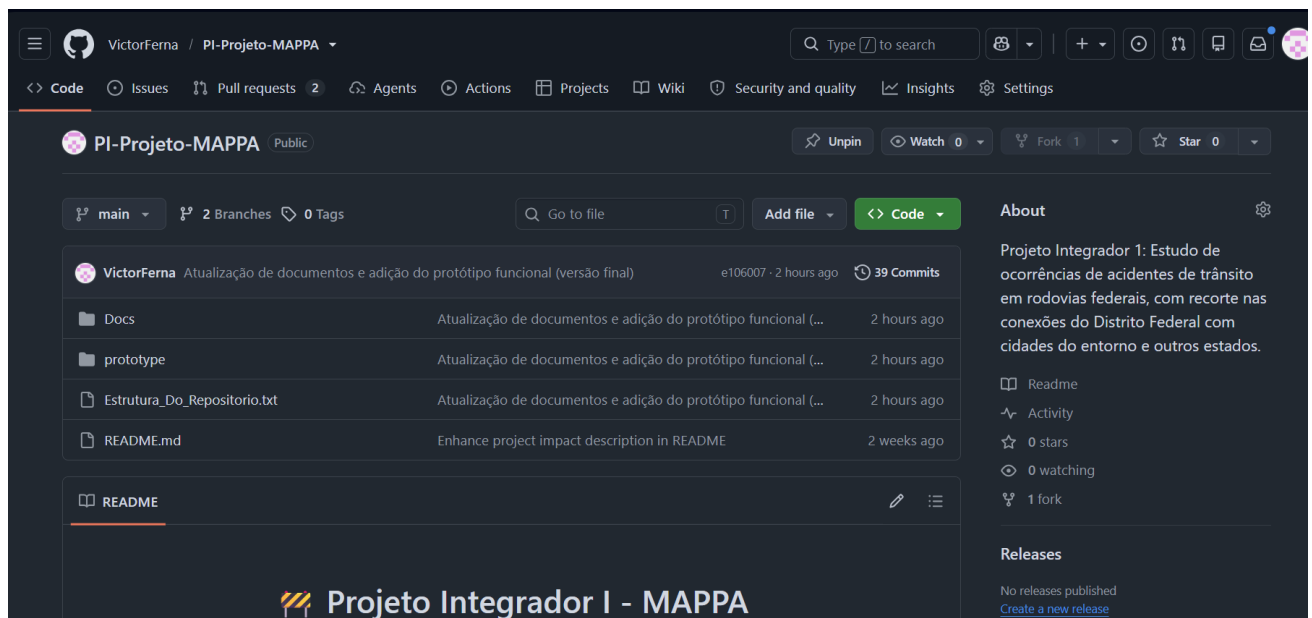
DOCUMENTO NA VERSÃO Nº1, DESENVOLVIDO POR GRUPO MAPPA

1. Apresentação do tutorial

Este documento tem como objetivo explicar, de forma simples e organizada, como baixar, preparar e executar o protótipo do projeto MAPPA em um computador local. O foco do tutorial é permitir que qualquer pessoa com conhecimentos básicos de computador e Visual Studio Code consiga reproduzir o funcionamento principal da solução.

Ao final deste guia, o leitor deverá ser capaz de:

- baixar o projeto no GitHub;
- organizar os arquivos necessários para o ETL;
- executar o script de tratamento de dados;
- abrir o dashboard em Streamlit;
- entender o fluxo geral do código e a lógica do protótipo.

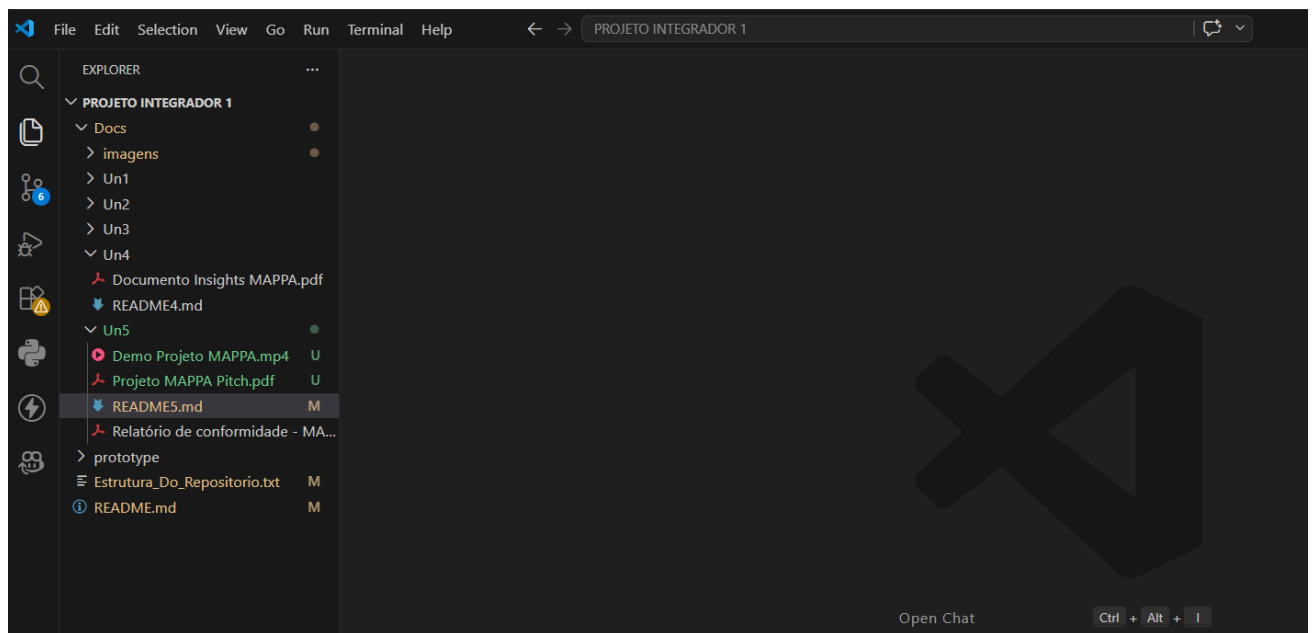


2. Pré-requisitos

Antes de iniciar, recomenda-se verificar se o ambiente possui os programas e componentes abaixo instalados.

- Visual Studio Code;
- Python 3 instalado e configurado no PATH;
- Git instalado, caso o download seja feito por clonagem do repositório;
- Acesso aos arquivos CSV da base da PRF que serão usados no ETL;
- Bibliotecas Python necessárias para ETL e dashboard, como pandas e streamlit.

Também é importante ter uma pasta local dedicada ao projeto, para evitar perda de arquivos ou confusão entre versões diferentes.

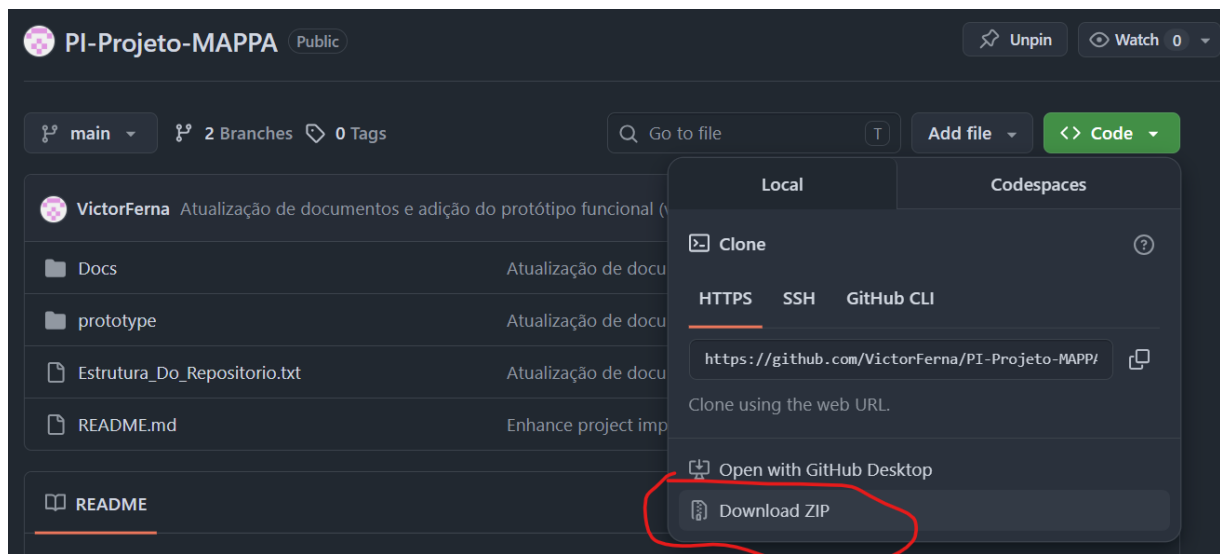


3. Como baixar o protótipo no GitHub

O primeiro passo é obter o projeto MAPPA diretamente do repositório oficial. Isso pode ser feito de duas formas: baixando o arquivo compactado pelo navegador ou clonando o repositório com Git.

3.1 Download pelo navegador

1. Acesse o repositório do projeto no GitHub.
LINK: <https://github.com/VictorFerna/PI-Projeto-MAPPA>
2. Clique no botão verde ou cinza identificado como Code.
3. Selecione a opção Download ZIP.
4. Após o download, extraia a pasta em um local de fácil acesso no computador.



3.2 Clonagem com Git

Se o computador já tiver o Git instalado, a forma mais prática é clonar o repositório pelo terminal do VSCode.

```
git clone https://github.com/VictorFerna/PI-Projeto-MAPPA.git
cd PI-Projeto-MAPPA
```

Depois disso, abra a pasta do projeto no VSCode. A estrutura local deverá conter o código-fonte, a pasta de documentos e os arquivos necessários para a execução do protótipo.



```
C:\Windows\System32\cmd.e  x  +  v
Microsoft Windows [versão 10.0.26200.8655]
(c) Microsoft Corporation. Todos os direitos reservados.

C:\Users\victo\Desktop\tutorial>git clone https://github.com/VictorFerna/PI-Projeto-MAPPA.git
```

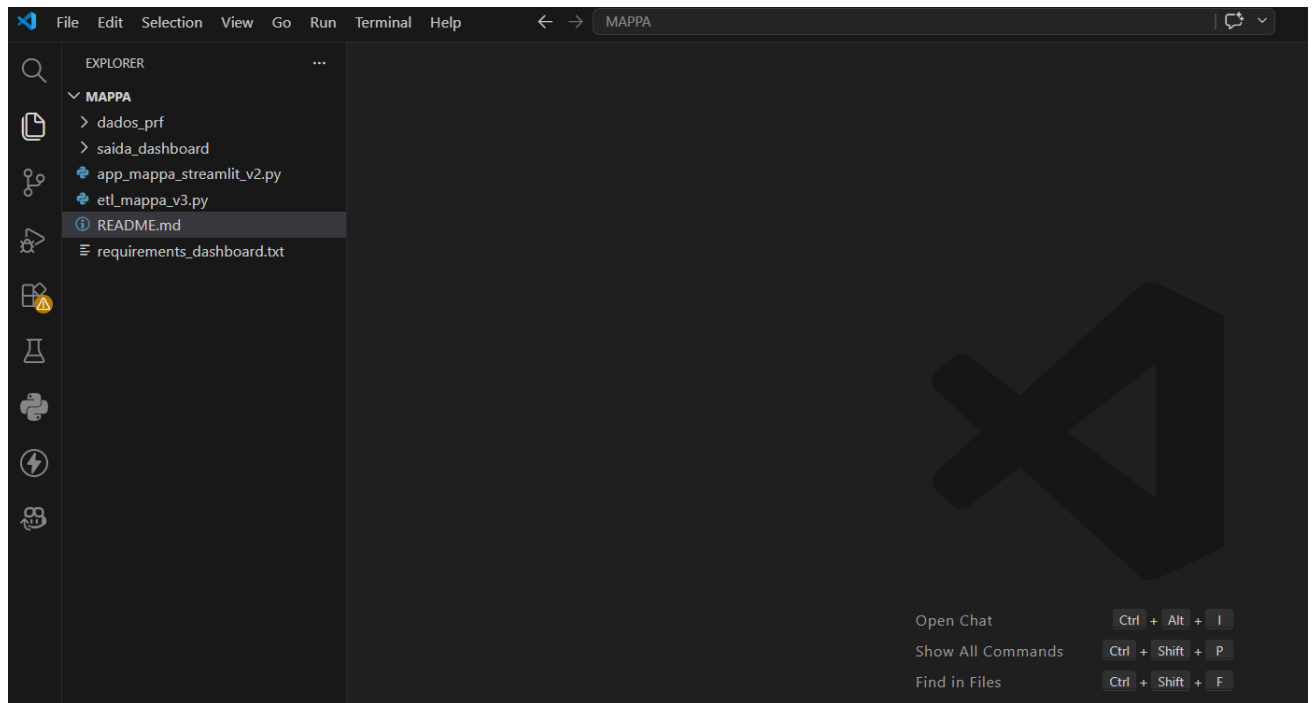
4. Organização básica da pasta do projeto

Depois de baixar o projeto, é importante conferir se a estrutura de pastas está organizada corretamente. O protótipo final do MAPPA trabalha com uma lógica simples: uma pasta de entrada com os CSVs da PRF, um script ETL, uma pasta de saída para os arquivos tratados e o aplicativo Streamlit para visualização.

Uma organização recomendada é a seguinte:

```
PI-Projeto-MAPPA/
├── Docs/
├── dados_prf/
├── saida_dashboard/
├── etl_mappa_v3.py
├── app_mappa_streamlit_v2.py
├── README.md
└── requirements_dashboard.txt
```

Caso alguma dessas pastas ainda não exista no seu computador, ela pode ser criada manualmente no próprio VSCode.

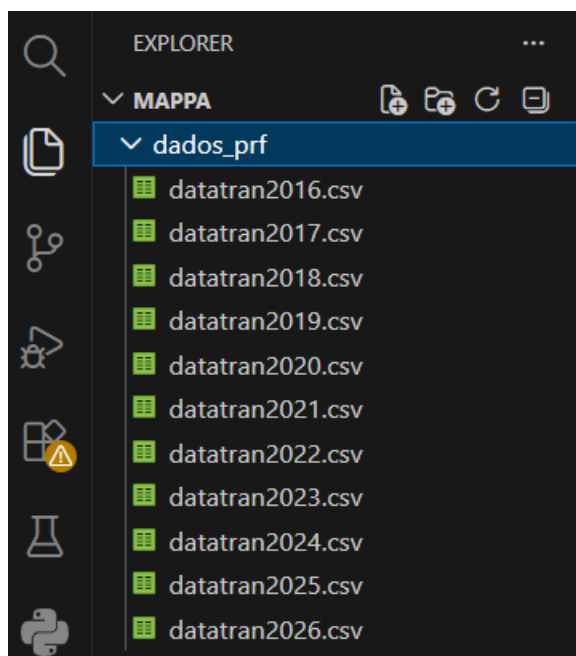


5. Como preparar os dados para o ETL

O ETL do MAPPA depende de arquivos CSV públicos com ocorrências de acidentes. Esses arquivos devem ser colocados dentro da pasta de entrada, normalmente nomeada como `dados_prf`.

Para que o processamento funcione corretamente, recomenda-se manter todos os arquivos na mesma pasta e com nomes que permitam identificar o ano, por exemplo: `datatran2016.csv`, `datatran2017.csv`, `datatran2018.csv` e assim por diante.

- Use arquivos compatíveis com a estrutura esperada pelo ETL.
- Evite renomeações aleatórias que removam o ano do nome do arquivo.
- Mantenha apenas os arquivos necessários para a análise.

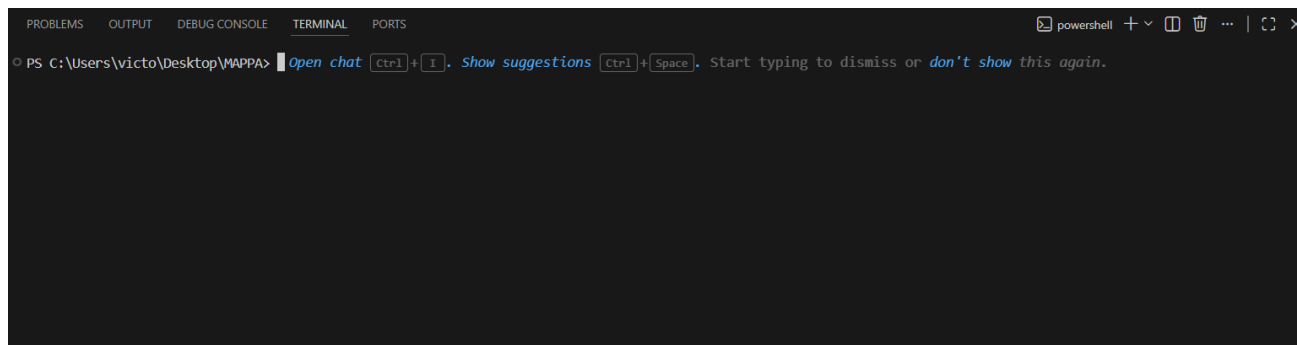


6. Como executar o ETL no VSCode

Com os arquivos de dados organizados, o próximo passo é executar o ETL. Esse processo é responsável por ler os CSVs brutos, aplicar o recorte do projeto, tratar inconsistências e gerar uma base pronta para AED, visualização e dashboard.

6.1 Abrindo o terminal

5. Abra a pasta do projeto no VSCode.
6. Clique em Terminal > New Terminal.
7. Confirme se o terminal foi aberto na pasta correta do projeto.



6.2 Executando o script

No terminal, execute o script principal do ETL.

```
python etl_mappa_v3.py
```

Durante a execução, o sistema deverá informar quantos arquivos foram encontrados, o volume da base bruta, o volume da base tratada e os arquivos CSV gerados na pasta `saida_dashboard`.

```

PS C:\Users\victo\Desktop\MAPPA> python etl_mappa_v3.py
=== MAPPA ETL v3 ===
Lendo arquivos em: C:\Users\victo\Desktop\MAPPA\dados_prf
Arquivos encontrados: 11
- datatran2016.csv
- datatran2017.csv
- datatran2018.csv
- datatran2019.csv
- datatran2020.csv
- datatran2021.csv
- datatran2022.csv
- datatran2023.csv
- datatran2024.csv
- datatran2025.csv
- datatran2026.csv

Base bruta concatenada: 752505 linhas x 32 colunas
C:\Users\victo\Desktop\MAPPA\etl_mappa_v3.py:132: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
  fallback = pd.to_datetime(s.loc[out.isna()], errors="coerce", dayfirst=True)
Base tratada (recorte do projeto): 48876 linhas x 44 colunas
Datas inválidas (NaT): 0
Municípios únicos: 109
Top municípios:

```

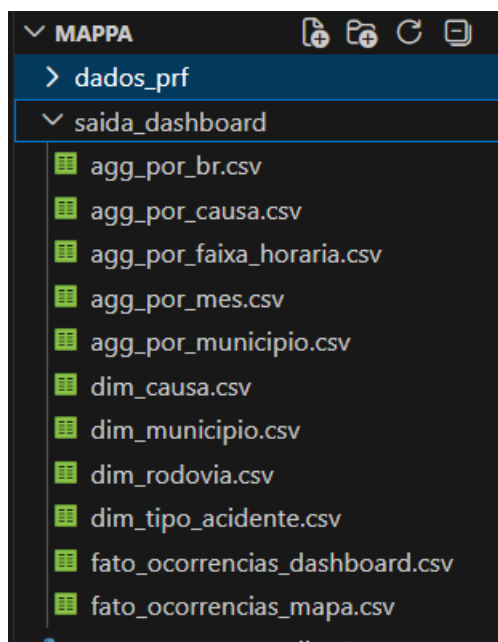
6.3 O que o ETL faz

Em termos simples, o ETL do MAPPA realiza quatro grandes tarefas:

- lê múltiplos arquivos CSV da PRF;
- padroniza colunas e ajusta datas;
- aplica o recorte geográfico e temático do projeto;
- gera fatos, dimensões e agregados para o dashboard.

Ao fim da execução, os principais arquivos de saída esperados são:

- fato_ocorrencias_dashboard.csv;
- fato_ocorrencias_mapa.csv;
- dim_causa.csv;
- dim_tipo_acidente.csv;
- dim_rodovia.csv;
- dim_municipio.csv;
- agg_por_br.csv;
- agg_por_mes.csv;
- agg_por_causa.csv;
- agg_por_faixa_horaria.csv;
- agg_por_municipio.csv.



7. Como executar o dashboard em Streamlit

Depois que o ETL finalizar, o dashboard pode ser iniciado. O Streamlit é a ferramenta usada para transformar os arquivos CSV tratados em uma interface interativa no navegador.

7.1 Instalação das dependências

Se for a primeira vez executando o projeto naquele computador, instale as bibliotecas necessárias.

```
pip install -r requerimentos_dashboard.txt
```

```
Concluído. Base pronta para AED, visualização e dashboard.
PS C:\Users\victo\Desktop\MAPPA> pip install -r requerimentos_dashboard.txt
Requirement already satisfied: streamlit<1.36,>=1.35 in c:\users\victo\appdata\local\programs\python\python312\lib\site-packages (from -r requerimentos_dashboard.txt (line 1)) (1.58.0)
Requirement already satisfied: pandas<2.0,>=1.5.3 in c:\users\victo\appdata\local\programs\python\python312\lib\site-packages (from -r requerimentos_dashboard.txt (line 2)) (2.0.3)
Requirement already satisfied: plotly<5.22,>=5.21 in c:\users\victo\appdata\local\programs\python\python312\lib\site-packages (from -r requerimentos_dashboard.txt (line 3)) (6.8.0)
Requirement already satisfied: numpy<1.24,>=1.23 in c:\users\victo\appdata\local\programs\python\python312\lib\site-packages (from -r requerimentos_dashboard.txt (line 4)) (2.0.4)
Requirement already satisfied: altair<5.4.0,!=5.4.1,<7,>=4.0 in c:\users\victo\appdata\local\programs\python\python312\lib\site-packages (from streamlit<1.36->-r requerimentos_dashboard.txt (line 1)) (6.2.1)
Requirement already satisfied: blinker<2,>=1.5.0 in c:\users\victo\appdata\local\programs\python\python312\lib\site-packages (from streamlit<1.36->-r requerimentos_dashboard.txt (line 1)) (1.9.0)
Requirement already satisfied: cachetools<8,>=5.5 in c:\users\victo\appdata\local\programs\python\python312\lib\site-packages (from streamlit<1.36->-r requerimentos_dashboard.txt (line 1)) (7.1.4)
Requirement already satisfied: click<9,>=7.0 in c:\users\victo\appdata\local\programs\python\python312\lib\site-packages (from streamlit<1.36->-r requerimentos_dashboard.txt (line 1)) (8.4.1)
Requirement already satisfied: gitpython<3.1.19,<4,>=3.0.7 in c:\users\victo\appdata\local\programs\python\python312\lib\site-packages (from streamlit<1.36->-r requerimentos_dashboard.txt (line 1)) (3.1.18)
```

7.2 Iniciando o app

Com as dependências instaladas e a pasta `saida_dashboard` preenchida, execute o aplicativo Streamlit.

```
streamlit run app_mappa_streamlit_v2.py
```

O terminal deverá exibir um endereço local, normalmente começando com `http://localhost`. Ao abrir esse endereço no navegador, o dashboard do MAPPA será carregado.


```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
streamlit
PS C:\Users\victo\Desktop\MAPPA> streamlit run app_mappa_streamlit_v2.py
2026-06-22 01:39:16.632 Uvicorn server started on 0.0.0.0:8501

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.1.16:8501

2026-06-22 01:39:20.396 Please replace `use_container_width` with `width`.

`use_container_width` will be removed after 2025-12-31.

For `use_container_width=True`, use `width='stretch'`. For `use_container_width=False`, use `width='content'`.
2026-06-22 01:39:20.420 Please replace `use_container_width` with `width`.
```

7.3 O que conferir ao abrir o dashboard

- Se os KPIs aparecem corretamente;
- Se os filtros respondem às seleções do usuário;
- Se os gráficos estão sendo exibidos;
- Se a aba do mapa carrega os pontos georreferenciados.



8. Como utilizar o protótipo

O protótipo foi pensado para facilitar a leitura dos dados por diferentes perfis de usuário. Por isso, a navegação é direta e baseada em filtros, métricas e visualizações.

8.1 Visão geral

Na visão geral, o usuário encontra os indicadores centrais do recorte selecionado, como número de acidentes, feridos, mortos, rodovias e municípios.

8.2 AED e gráficos

Na parte analítica, o usuário pode observar a distribuição dos dados, entender as causas mais recorrentes e acompanhar a evolução temporal dos registros.

8.3 Mapa

A visualização geográfica permite identificar espacialmente os pontos das ocorrências, tornando a análise mais intuitiva.

8.4 Filtros

Os filtros por ano, rodovia, município, severidade e causa permitem adaptar a leitura do painel para diferentes perguntas de análise.

MAPPA

Dashboard + AED em Streamlit

Pasta de dados

saida_dashboard

Filtros

Ano

2016 × 2017 × 2018 ×

2019 × 2020 × 2021 ×

2022 × 2023 × 2024 ×

2025 × 2026 ×

Rodovia

BR-020 × BR-040 ×

BR-060 × BR-070 ×

BR-080 ×

Município

Choose options

Severidade

COM FERIDOS ×

COM MORTOS ×

SEM VITIMAS ×

Causa

9. Explicação simples do código

Para uma pessoa leiga, o funcionamento do código do MAPPA pode ser entendido em três blocos principais.

9.1 Bloco 1 — Leitura dos dados

O código começa abrindo os arquivos CSV da PRF. Esses arquivos são como planilhas grandes com informações brutas sobre acidentes.

9.2 Bloco 2 — Tratamento e organização

Depois da leitura, o sistema limpa os dados, padroniza nomes, ajusta datas, seleciona o recorte de interesse e cria novas colunas úteis para análise.

9.3 Bloco 3 — Visualização

Com a base tratada, o dashboard em Streamlit lê os arquivos gerados e transforma os dados em gráficos, indicadores e mapas.

Uma maneira simples de explicar isso é a seguinte:

- o dado bruto é a matéria-prima;
- o ETL é o processo que organiza essa matéria-prima;
- o dashboard é a vitrine final, onde a informação aparece de forma clara.

CÓDIGO ETL

```
etl_mappa_v3.py X
etl_mappa_v3.py > ...
30
31 def log(msg: str) -> None:
32     print(msg, flush=True)
33
34
35 def normalizar_texto(txt: str) -> str:
36     txt = unicodedata.normalize("NFKD", str(txt)).encode("ascii", "ignore").decode("utf-8")
37     txt = txt.lower().strip()
38     txt = re.sub(r"^[a-z0-9]+", "_", txt)
39     return txt.strip("_")
40
41
42 def detectar_sep(path: Path) -> str:
43     with path.open("r", encoding="latin1", errors="ignore") as f:
44         sample = f.read(4096)
45     try:
46         dialect = csv.Sniffer().sniff(sample, delimiters=[";", " ", "\t"])
47         return dialect.delimiter
48     except Exception:
49         return ";" if sample.count(";") >= sample.count(" ") else " "
50
51
52 def ler_csv(path: Path) -> pd.DataFrame:
53     sep = detectar_sep(path)
54     try:
55         return pd.read_csv(path, sep=sep, encoding="latin1", low_memory=False)
56     except UnicodeDecodeError:
57         return pd.read_csv(path, sep=sep, encoding="utf-8", low_memory=False)
58
59
60 def padronizar_colunas(df: pd.DataFrame) -> pd.DataFrame:
61     df = df.copy()
62     df.columns = [normalizar_texto(c) for c in df.columns]
63
64     aliases = {
65         "id_ocorrendia": "id",
66         "data": "data_inversa",
```

CÓDIGO STREAMLIT

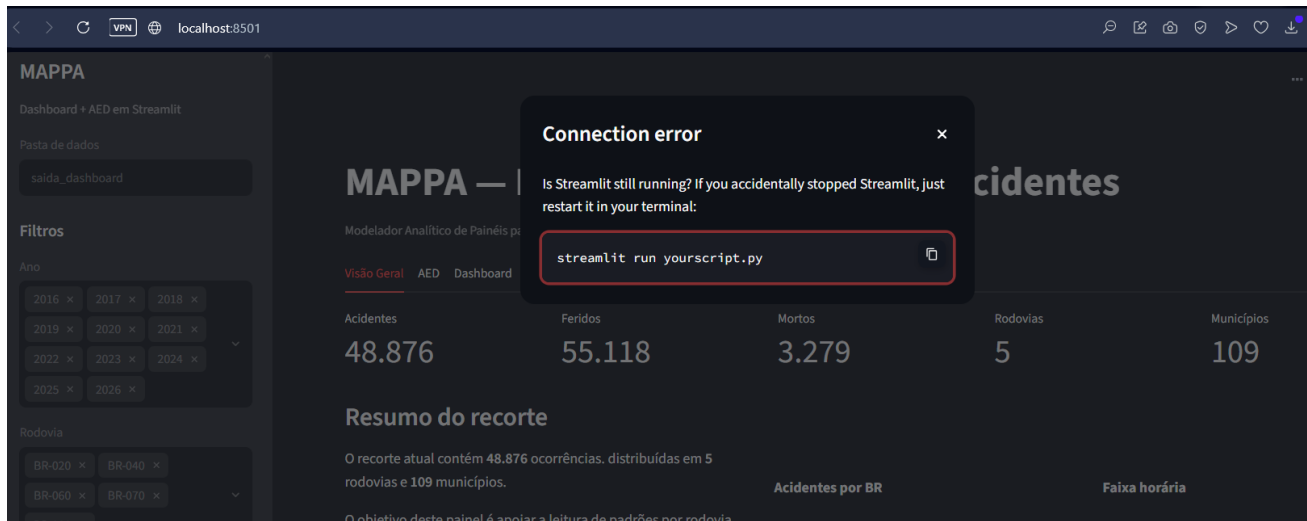
```
app_mappa_streamlit_v2.py X
app_mappa_streamlit_v2.py > ...
1
2 from __future__ import annotations
3
4 import io
5 from pathlib import Path
6
7 import numpy as np
8 import pandas as pd
9 import plotly.express as px
10 import plotly.graph_objects as go
11 import streamlit as st
12
13 # =====
14 # MAPPA Dashboard + AED
15 # -----
16 # Painel Streamlit para o projeto MAPPA
17 # Consome a saída do etl_mappa_v3.py
18 # Entrada esperada: pasta "saida_dashboard" com os CSVs gerados
19 # =====
20
21 st.set_page_config(
22     page_title="MAPPA – Dashboard de Acidentes",
23     page_icon="🚗",
24     layout="wide",
25     initial_sidebar_state="expanded",
26 )
27
28 DEFAULT_DATA_DIR = "saida_dashboard"
29
30
31 # -----
32 # Utilidades
33 # -----
34 @st.cache_data(show_spinner=False)
35 def load_csv(path: Path) -> pd.DataFrame:
36     return pd.read_csv(path, encoding="utf-8-sig")
37
```

10. Dicas e solução de problemas

Caso o protótipo não execute corretamente, alguns pontos devem ser verificados:

- confirmar se o Python está instalado;
- confirmar se o terminal está aberto na pasta correta do projeto;
- verificar se os CSVs estão dentro da pasta dados_prf;
- executar primeiro o ETL e só depois o Streamlit;
- confirmar se a pasta saida_dashboard foi gerada corretamente;
- verificar se as dependências do requirements_dashboard.txt foram instaladas.

Se o dashboard abrir sem dados, o problema normalmente está na ausência dos arquivos tratados na pasta de saída. Se o ETL falhar, o problema geralmente está na pasta de entrada, no nome dos arquivos ou na ausência de bibliotecas.



11. Considerações finais

O tutorial apresentado mostra como o MAPPA pode ser executado localmente de forma simples, desde o download no GitHub até a visualização do dashboard em funcionamento. A proposta do projeto é transformar dados públicos em informação analítica acessível, apoiando leitura, interpretação e tomada de decisão.

Mesmo sendo um protótipo acadêmico, o sistema já demonstra uma estrutura consistente de ingestão, tratamento e visualização. Por isso, entender esse fluxo é essencial para compreender o valor do projeto como solução aplicada à segurança viária.



12. Anexo — Comandos principais

Para facilitar a consulta rápida, os comandos centrais do projeto estão resumidos abaixo.

```
git clone https://github.com/VictorFerna/PI-Projeto-MAPPA.git
cd PI-Projeto-MAPPA
pip install -r requirements_dashboard.txt
python etl_mappa_v3.py
streamlit run app_mappa_streamlit_v2.py
```